AI Engineering Interview preparation guide

Last updated: 4/24/2026

We aim to hire engineers who:

- Solve problems, not just write code
- Think like product owners
- Leverage AI to accelerate development 10X
- Raise the overall engineering bar

We are not just hiring coders. We are hiring engineers who build products and deliver outcomes.

Understanding Engineer vs Product Engineer

Traditional Engineer	Product Engineer
Implements requirements	Understands the problem and user
Focuses on technical correctness	Focuses on outcomes and value
Waits for specification	Helps define the solution
Optimizes code	Optimizes user experience and impact
Works within scope	Expands scope if it improves product

Interview Format:

Round	Focus	Duration
Round 1	**AI Coding round**	60 Mins
Round 2	Problem Solving + System Design	60 Mins
Round 3	Product thinking + Challenging Project deep dive / Design + Behavioural	60-90 Mins
Round 4	Hiring Manager Round [Fit + Behavioural + Growth]	45 Mins
Round 5	Leadership Interview	30 Mins

AI Coding Round:

The interviewer may ask the candidate for a coding problem on an existing code repository or a completely new project from scratch.

The candidate will be asked to share their screen along with camera. The screen will be recorded.

1. **Please make sure you have an AI coding assistant installed on your machine, preferably in Agent Mode.**
2. **If you do not have anything on your machine, we suggest you use Antigravity from Google or GitHub Copilot**
3. **We suggest you spend enough time getting familiar with the coding tool you are planning to use during the coding round.**
4. **You will be asked to share your screen along with camera. The screen will be recorded.**

Samples:

- Design a backend service that returns relevant documents for a query.
- Design a system that ingests documents and prepares them for search.
- Track AI responses and user feedback.

Best Practices:

Requirements First: Clarify ambiguity before prompting. Use Plan mode or ask first mode to brainstorm

Orchestration: Break problems into modular prompts.

Critical Review: Review everytime and run the code, remove noise.

Verification: Proactively write tests and validate edge cases.

Ownership: Able to understand and reason every line of code